LSA e Busca Híbrida

Aula 07 — Vetores densos pela SVD e fusão de rankings

Projeto Integrador III ▪ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

De onde viemos, para onde vamos

Fase 2 (fechada na Aula 06)

- Modelos **léxicos**: TF-IDF, BM25
- Métricas: P@k, MAP, nDCG, MRR
- Rocchio e *pseudo*-realimentação

Hoje (abre a Fase 3)

- Casar por **significado**, não por palavra
- **Embeddings**: vetores densos
- LSA: a SVD da TF-IDF, no R
- Busca **híbrida** (RRF)

Ideia

"carro" e "automóvel" não têm nenhuma palavra em comum — para o BM25, nenhuma relação. Hoje vemos o mesmo problema **nos nossos 8 documentos**.

Ponto de partida: os 8 documentos e a matriz TF-IDF

```r
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "o indice invertido acelera a busca em muitos documentos",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
tok    <- function(x) unlist(strsplit(tolower(x), "\\s+"))
tokens <- lapply(docs, tok)
vocab  <- sort(unique(unlist(tokens)))
tf  <- sapply(tokens, function(t) as.integer(table(factor(t, levels = vocab))))
rownames(tf) <- vocab
idf <- log(ncol(tf) / rowSums(tf > 0))
w   <- tf * idf                          # 45 termos x 8 documentos
```

O problema: descasamento de vocabulário

```
cosseno <- function(a, b) {
  den <- sqrt(sum(a^2)) * sqrt(sum(b^2)); if (den == 0) 0 else sum(a*b) / den }
vq <- function(texto) as.integer(table(factor(tok(texto), levels = vocab))) * idf
lexico <- function(texto)
  sort(apply(w, 2, function(d) cosseno(vq(texto), d)), decreasing = TRUE)
round(lexico("busca"), 3)

   d5    d7    d1    d2    d3    d4    d6    d8
0.270 0.265 0.000 0.000 0.000 0.000 0.000 0.000

round(lexico("ranqueamento"), 3)

   d3    d1    d2    d4    d5    d6    d7    d8
0.413 0.000 0.000 0.000 0.000 0.000 0.000 0.000
```

- "busca": só d5 e d7 têm a palavra. d1 ("recuperação de informação...") fala do mesmo assunto e recebe **zero**.
- "ranqueamento": só d3. d1 ("ordena documentos por relevância") é um sinônimo e também fica com **zero**.

Esparso × denso

```
dim(w)

[1] 45  8

sum(w[, "d1"] != 0)          # termos com peso em d1

[1] 7
```

Esparso (até aqui)

Uma dimensão por **termo**: d1 tem peso em 7 das 45. Dois textos sem termo em comum têm cosseno **zero**.

Denso (hoje)

Poucas dimensões (2, 20, 100...), **todas** com valor. Cada dimensão é um eixo aprendido dos dados, não uma palavra. Hoje, o denso é o **LSA**.

"You shall know a word by the company it keeps" (Firth, 1957): palavras que aparecem nos **mesmos contextos** ganham vetores **próximos**. Esse vetor é o **embedding**.

LSA: o primeiro embedding é uma SVD

$$\underbrace{W}_{45\times8} = \underbrace{U}_{45\times8}\,\underbrace{\Sigma}_{8\times8}\,\underbrace{V^{\top}}_{8\times8} \qquad \Longrightarrow \qquad W \approx U_k\,\Sigma_k\,V_k^{\top}$$

W	a matriz TF-IDF (`w`): termos $\times$ documentos
U	cada coluna é uma **direção** latente: uma combinação de termos (linhas = termos)
Σ	diagonal com os **valores singulares** $\sigma_1 \geq \sigma_2 \geq \cdots$: a força de cada direção
V	quanto cada **documento** usa cada direção (linhas = documentos)
k	quantas direções guardamos; o documento vira as k coordenadas da linha de $V_k\Sigma_k$
energia	$\sum_{i\leq k}\sigma_i^2 / \sum_i \sigma_i^2$: a fração de W que k direções explicam

LSA = *Latent Semantic Analysis* (Deerwester et al., 1990; IIR cap. 18). Tudo em R base: `svd()`.

A SVD no R

```
s <- svd(w)                                  # w = s$u %*% diag(s$d) %*% t(s$v)
round(s$d, 3)                                # valores singulares

[1] 5.602 5.313 5.022 4.783 4.672 4.522 4.319 4.024

round(cumsum(s$d^2) / sum(s$d^2), 3)         # energia acumulada

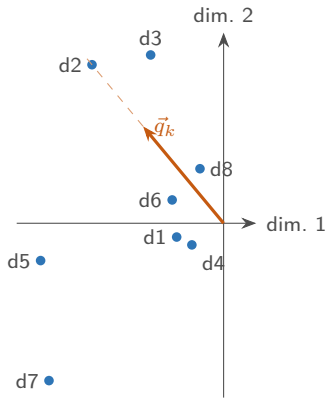
[1] 0.170 0.323 0.459 0.583 0.701 0.811 0.912 1.000
```

- Com $k = 2$ guardamos só **32%** da energia; seriam precisas 7 dimensões para passar de 90%.
- Os valores singulares quase não caem (5,602 … 4,024): com 8 frases curtas, o *corpus* **não tem estrutura forte**. A lição vale mais pela mecânica do que pelo resultado.

Os 8 documentos em 2 dimensões

```
k  <- 2
Uk <- s$u[, 1:k]
D  <- s$v[, 1:k] %*% diag(s$d[1:k])
rownames(D) <- colnames(w)
round(D, 3)            # V_k Sigma_k
```

```
     [,1]   [,2]
d1 -0.862 -0.253
d2 -2.417  2.910
d3 -1.340  3.089
d4 -0.583 -0.397
d5 -3.361 -0.685
d6 -0.945  0.428
d7 -3.207 -2.888
d8 -0.434  1.001
```



Seta: direção da consulta "modelo de recuperacao" (próximo slide).

Os **sinais** podem sair trocados em outra máquina (outra biblioteca de álgebra linear). Os **cossenos não mudam**.

Dobrar a consulta para o espaço de k dimensões

$$\vec{q}_k \;=\; U_k^{\top}\,\vec{q}_0 \qquad (2\times 45)\,(45\times 1) = (2\times 1)$$

$\vec{q}_0$ é o vetor TF-IDF da consulta (Aula 02); $U_k^{\top}$ o projeta nas k direções.

```
q0 <- vq("modelo de recuperacao")
qk <- t(Uk) %*% q0
round(qk, 3)

       [,1]
[1,] -0.370
[2,]  0.445
```

A consulta cai na mesma direção de `d2` $(-2{,}417;\ 2{,}910)$: cosseno ≈ 1. Depois, **cosseno** entre $\vec{q}_k$ e cada linha de `D` — a mesma conta da Aula 02, com 2 dimensões em vez de 45.

Ranking LSA ($k = 2$) e métricas

```
r_lsa <- sort(apply(D, 1, function(d) cosseno(qk, d)), decreasing = TRUE)
round(r_lsa, 3)

    d2     d3     d8     d6     d5     d1     d4     d7
 1.000  0.960  0.960  0.900  0.474  0.397  0.096 -0.039

round(metricas(r_lsa, g), 3)      # gabarito da Aula 05, limiar 2

   P_at_k        AP        RR  nDCG_bin nDCG_grad
    0.667     1.000     1.000     1.000     0.966
```

- Os dois relevantes (d2, d3, grau 2) nas posições 1 e 2: **AP = 1**. O BM25 tinha AP = 0,833.
- d4 ("... recuperação moderna", grau 0), que incomodava na Aula 06, cai para o 7º lugar.
- d8 (grau 0) empata com d3 em 0,960: o LSA também aproxima o que **não** devia.

k é um hiperparâmetro

```
rank_lsa <- function(texto, k) {
  D  <- s$v[, 1:k, drop = FALSE] %*% diag(s$d[1:k], k); rownames(D) <- colnames(w)
  qk <- t(s$u[, 1:k, drop = FALSE]) %*% vq(texto)
  sort(apply(D, 1, function(d) cosseno(qk, d)), decreasing = TRUE)
}
m <- sapply(2:4, function(k) metricas(rank_lsa("modelo de recuperacao", k), g))
colnames(m) <- paste0("k=", 2:4)
round(m, 3)

            k=2   k=3   k=4
P_at_k    0.667 0.333 0.667
AP        1.000 0.750 1.000
RR        1.000 1.000 1.000
nDCG_bin  1.000 0.877 1.000
nDCG_grad 0.966 0.925 1.000
```

$k = 2$: d2 1,000 d3 0,960 d8 0,960 d6 0,900 ...
$k = 3$: d3 0,913 d6 0,863 d8 0,741 d2 0,710 ...
$k = 4$: d3 0,861 d2 0,751 d1 0,559 d6 0,537 ...

O resultado **oscila** com k (AP: 1 $\rightarrow$ 0,75 $\rightarrow$ 1). Como k_1, b e β: escolher no **dev**, relatar no **teste** uma vez.

Os riscos: relações inventadas

```
round(rank_lsa("busca", 2), 3)

    d4     d7     d1     d5     d6     d2     d3     d8
 0.999  0.982  0.967  0.942  0.566  0.149 -0.134 -0.134

round(rank_lsa("ranqueamento", 2), 3)

    d8     d3     d2     d6     d5     d1     d4     d7
 0.999  0.999  0.949  0.715  0.170  0.086 -0.224 -0.353
```

- "busca": d4 e d1 **não têm a palavra** e ganham 0,999 e 0,967. Ganho de *recall* (d1) ou ruído (d4)?
- "ranqueamento": d8 ("ciência de dados...") **empata no topo** com d3. Com 8 frases, o LSA inventa relações.

Ganho e ruído vêm do mesmo mecanismo

Em 2 dimensões a consulta fica perto de muitos documentos: d5, que tem "busca", cai para 4º.

Por que d8? Relações inventadas com corpus pequeno

```
intersect(tokens$d3, tokens$d8)                    # o que tem em comum

[1] "e"  "de"

round(c(tfidf = cosseno(w[, "d3"], w[, "d8"]),     # 45 dimensoes
        lsa2  = cosseno(D["d3", ], D["d8", ])), 3) # 2 dimensoes

tfidf  lsa2
0.058 1.000

round(rank_lsa("ranqueamento", 4), 3)              # com k = 4

    d3     d6     d8     d1     d2     d4     d7     d5
 0.998  0.720  0.534  0.392  0.317  0.204  0.057 -0.246
```

- Só e e de, palavras sem assunto: cosseno 0,058 na TF-IDF, **1** em 2 dimensões. Com 8 frases, a SVD não separa "fala de ranqueamento" de "usa de".
- k decide entre ganho e ruído: com $k = 4$, d3 volta ao topo; com $k = 8$ (sem compressão) volta o léxico. Escolhe-se no **dev**.

Esparso, denso — e híbrido: RRF

Esparso (BM25)

Exato em termos raros, nomes, códigos. Interpretável.

Denso (LSA)

Sinônimos e paráfrases. Pode inventar relações.

Reciprocal Rank Fusion

$$\mathrm{RRF}(d) = \sum_{r} \frac{1}{c + \mathrm{pos}_r(d)}, \qquad c = 60$$

- Usa só as **posições**: não importa que BM25 e cosseno estejam em escalas diferentes.
- Conta à mão, d3: 1º no BM25, 2º no LSA

$$\tfrac{1}{61} + \tfrac{1}{62} = 0{,}01639 + 0{,}01613 = 0{,}0325$$

RRF no R: BM25 + LSA

```
rrf <- function(..., c = 60) {
  rs <- list(...); ids <- unique(unlist(lapply(rs, names)))
  s  <- sapply(ids, function(d) sum(sapply(rs, function(r) {
          p <- which(names(r) == d); if (length(p)) 1 / (c + p) else 0 })))
  sort(s, decreasing = TRUE) }
q <- "modelo de recuperacao"
r_bm25 <- ranking_bm25(q)
r_hib  <- rrf(r_bm25, r_lsa)
round(r_hib, 4)

    d3     d2     d1     d8     d6     d4     d5     d7
0.0325 0.0323 0.0313 0.0313 0.0308 0.0306 0.0303 0.0294

round(rbind(bm25 = metricas(r_bm25, g), cosseno = metricas(lexico(q), g),
            lsa = metricas(r_lsa, g), rrf = metricas(r_hib, g)), 3)

        P_at_k    AP  RR nDCG_bin nDCG_grad
bm25     0.667 0.833 1.0    0.920     0.951
cosseno  0.333 0.500 0.5    0.651     0.837
lsa      0.667 1.000 1.0    1.000     0.966
rrf      0.667 1.000 1.0    1.000     0.990
```

d1 (0,03128) e d8 (0,03126) quase empatam: o RRF ordena, não mede distância. O híbrido tem o melhor nDCG

Para casa

Tarefa (Parte D)

1. `lsa(ix, k)` no *corpus* do grupo; comparar o top-5 de `ranking_lsa` com o do BM25 numa consulta, lendo os textos.
2. MAP de BM25, LSA com alguns k (10, 20, 50, até o número de documentos) e `rrf(BM25, LSA)`: escolher modelo e k no **dev**, relatar no **teste** uma vez.
3. Registrar a decisão: `modelo` (`"bm25"`, `"lsa"`, `"hibrido"`), `lsa_k` e `rrf_c` no `config.R`; ficha do projeto.

Próxima aula

- Reordenar o topo (*retrieve & rerank*)
- Proximidade e ordem das palavras da consulta

Leitura

- Manning et al., *IIR* — cap. 18 (LSA)
- Deerwester et al. (1990); Cormack, Clarke & Büttcher (2009), RRF

Obrigado!

Dúvidas? Vamos ao R.